



MiniVim README

by 2026JohnClass

作者: PhantomPhoenix

组织: Labmem

时间: September 30, 2026

版本: 1.0

邮箱: logic_1729@sjtu.edu.cn

既对沧海一声笑，何不潇洒走一回

目录

第 1 章 引言	1
1.1 为什么我们现在仍然要学习 Vim	1
1.2 第一次使用 Vim	2
1.2.1 Vim 的启动	2
1.2.2 模式与基础操作	2
1.2.2.1 插入文本	2
1.2.2.2 模式的查看	3
1.2.2.3 模式的切换	4
1.2.3 常见的编辑命令	5
1.2.3.1 移动光标 (hjkl)	5
1.2.3.2 删除字符	5
1.2.3.3 撤销操作	6
1.2.3.4 重做操作	7
1.2.4 寻求帮助	7
第 2 章 MiniVim	9
2.1 项目初衷	9
2.2 项目架构	9
2.2.1 框架与目录结构	9
2.2.2 src 内各文件简要介绍	10
2.2.3 建议的阅读与实现顺序	10
2.2.4 项目的运作流程	11
2.3 项目开发常识	11
2.3.1 编译与运行	11
2.3.2 Basic 部分的 Debug	12
2.4 项目测评	12
2.4.1 项目测评工具 vtemu 的使用与介绍	12
2.4.1.1 工具作用与准备	12
2.4.1.2 脚本记号与采样	12



2.5	项目实施要求	12
2.5.1	Basic 部分要求	13
2.5.1.1	Basic 概述	13
2.5.1.2	Normal 模式 (Basic)	13
2.5.1.3	Insert 模式 (Basic)	13
2.5.1.4	Command-line 模式 (Basic)	13
2.5.2	Advanced 部分要求	14
2.5.2.1	光标移动	14
2.5.2.2	文本编辑	14
2.5.2.3	模式切换	14
2.5.2.4	文件操作	15
2.5.2.5	窗口操作	15
2.5.2.6	评分标准	15
2.5.3	Extra 部分要求	17
第 3 章	项目前置知识	18
3.1	按键记号约定	18
第 4 章	光标的移动 (一)	19
4.1	上下左右移动:hjkl	19
第 5 章	光标的移动 (二)	20
第 6 章	Command-line 模式与文件操作	21
6.1	Command-line 模式	21
6.2	基本按键	21
6.2.1	printable ascii(Basic)	21
6.2.2	<CR>(Basic)	21
6.2.3	<Esc>(Basic)	21
6.3	文件操作	22
6.3.1	:wq [file](Basic)	22
6.3.2	:q!, :quit!(Basic)	22
6.4	Command-line Mode 显示要求	22
第 7 章	文本的修正与移动 (一)	23
7.1	插入	23
7.1.0.1	i	23
7.1.0.2	a	23
7.1.0.3	{printable ascii}	23
7.1.0.4	<Tab>	23
7.1.0.5	<BS>	23
7.1.0.6	<CR>	23
7.1.0.7	<Esc>	24

第 1 章 引言

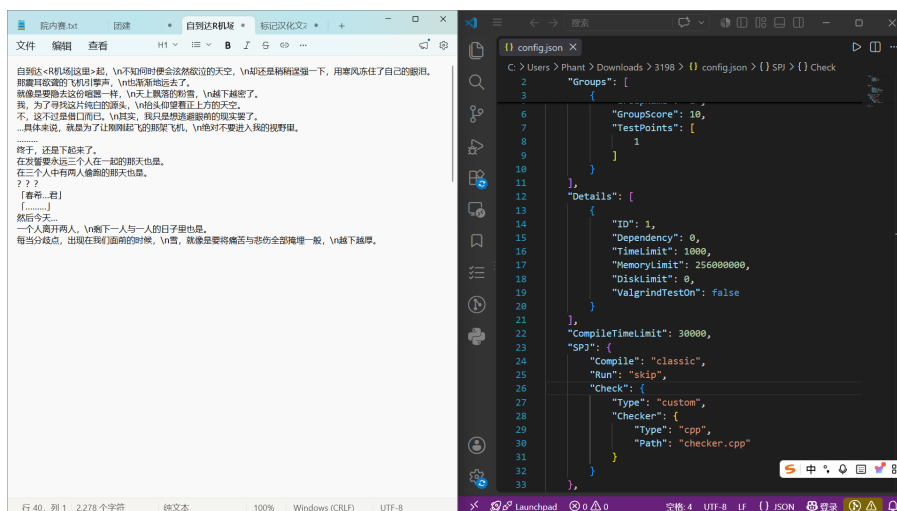
1.1 为什么我们现在仍然要学习 Vim

在学习 Vim 之前,我们先考虑一个看起来显然的问题:

什么是一个文本编辑器?

作为程序员,我们每天其实都在和“文本”打交道。源代码是文本,配置文件是文本,Shell 脚本是文本,Markdown 文档也是文本;

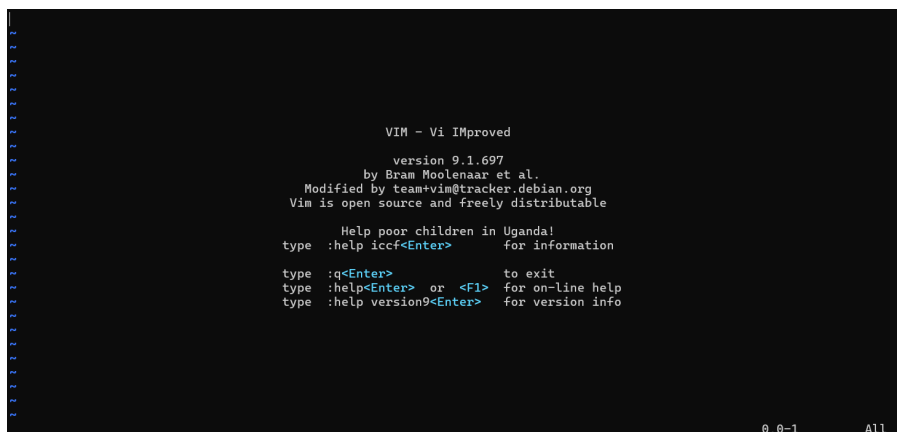
所谓文本编辑器 (Text Editor),就是一个用来查看和修改纯文本文件的程序。你大概已经使用过很多文本编辑器了。VS Code, Vim, Neovim, Notepad, 乃至很多 IDE, 其核心都包含一个文本编辑器。



那 Vim 作为一个基于命令行的文本编辑器,它的最主要的优势源于其被广泛认可的设计哲学:

- 编程时,你大部分时间花在阅读和编辑上,而非书写。正因如此,
- Vim 是一款模态编辑器:它针对插入文本和操作文本设有不同的模式。
- Vim 是可编程的(支持 Vimscript, 也支持 Python 等其他语言),而且 Vim 的界面本身也是一种编程语言;按键(带有助记名称)就是命令,而这些命令具有可组合性。
- Vim 避免使用鼠标,因为鼠标操作太慢;

- Vim 甚至避免使用方向键, 因为那也需要过多的手部移动. 最终的结果是, 这款编辑器能够跟上你思考的速度.



所以我们来学习如何启动和使用 Vim .

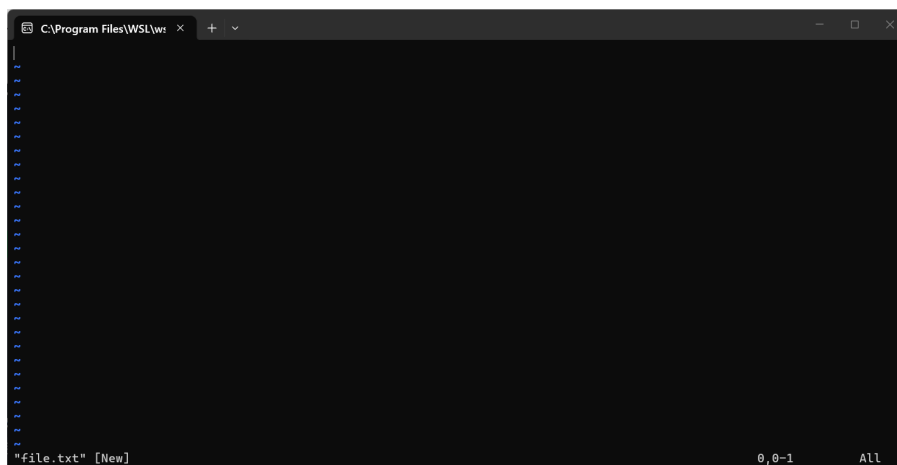
1.2 第一次使用 Vim

1.2.1 Vim 的启动

启动 Vim 的命令如下:

```
vim file.txt
```

在你的 Linux 系统中键入这一行, 现在 Vim 开始编辑一个名为 file.txt 的文件了。由于这是一个新建文件, 你会得到一个空的窗口。屏幕看起来会像下面这样:



以波纹线 `O` 开头的行表示该行在文件中不存在。换句话说, 如果 Vim 打开的文件不能充满这个显示的屏幕, 它就会显示以波纹线开头的行。在屏幕的底部, 有一个消息行指示文件名为 file.txt 并且说明这是一个新建的文件。这行信息是临时的, 新的信息可以覆盖它。

1.2.2 模式与基础操作

1.2.2.1 插入文本

Vim 是一个多模式的编辑器。就是说, 在不同模式下, 编辑器的响应是不同的。在普通模式下, 你敲入的字符只是命令; 而在插入模式, 你敲入的字符就成为插入的文本了。

当你刚刚进入 Vim，它处在 Normal Mode (普通模式, 下文不再重复)。通过敲入”i”命令(i 是插入 (Insert) 的缩写)可以启动 Insert Mode (插入模式, 下文不再重复)，这样你就可以输入文字了，这些文字将被插入到文件中。不用担心输错了，你随后还能够修正它。要输入下文的程序员打油诗，你可以这样敲：

iA very intelligent turtle

Found programming UNIX a hurdle

输入”turtle”后，按回车开始一个新行。直到输入全部文字。这时，Vim 会在窗口的底部显示 -INSERT- (中文模式显示的是 -插入-)，这表示你在 Insert Mode。

```
C:\Program Files\WSL\wn x + v  
A very intelligent turtle  
Found programming UNIX a hurdle  
  
-- INSERT --
```

2 / 31 All

最后，你输入 Esc 键退出 Insert Mode 而回到 Normal Mode，同时左下角的“-INSERT”也消失了，现在在你的 Vim 窗口中就有两行文字了：

```
A very intelligent turtle
Found programming UNIX a hurdle
```

2/30 All

1.2.2.2 模式的查看

在 Vim 中当然提供了不止一种模式，除去上文所提到的 Normal Mode 和 Insert Mode, 还有 Visual Mode, Replace Mode 和 Command Mode。他们分别起到选择文本、替换文本、键入命令的特殊作用。在 Vim 的界面中，默认左下角会显示当前所处的模式，无显示即为 Normal Mode。

同时，在不同的操作模式下，同样的按键会产生不同的效果。

例如, 字母 x 在 Insert Mode 下只是输入字符 x, 但在 Normal Mode 下会删除光标所在的字符, 而在 Visual Mode 下则会删除当前选中的内容, 这些具体的差异我们将会在后面的行文中具体展现.

1.2.2.3 模式的切换

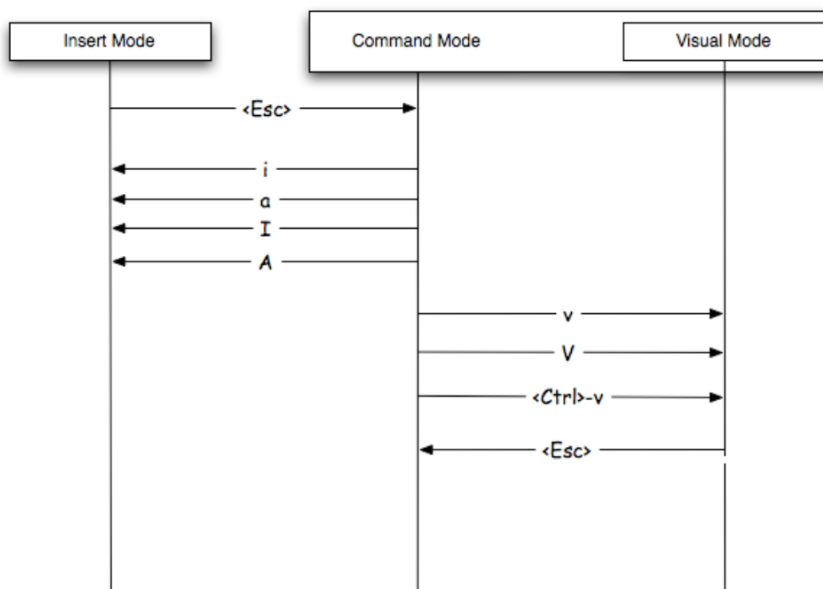
按 <ESC> (退出键) 可以从任何模式返回到 Normal Mode; 为此, 因为其极其常用, 可以考虑将 Caps Lock 键映射为 Escape, 或者用一个简单的按键序列为 <ESC> 创建替代映射方案.

从 Normal Mode 出发, 按 i 进入 Insert Mode, 按 R 进入 Replace Mode, 按 v 进入 Visual Mode, 按 V 进入 Visual Line Mode, 按 : 进入 Command Mode.

对于 Normal Mode 到 Insert Mode 的例子, 这是日常应用中最常见的情形, 比如:

- 在下方插入一行: o
- 在上方插入一行: O
- 在当前光标后插入: a
- 在当前光标前插入: i
- 在行尾插入: A
- 在行首插入: I

我们将常见情况整理成了一张图, 大家可以体会一下:



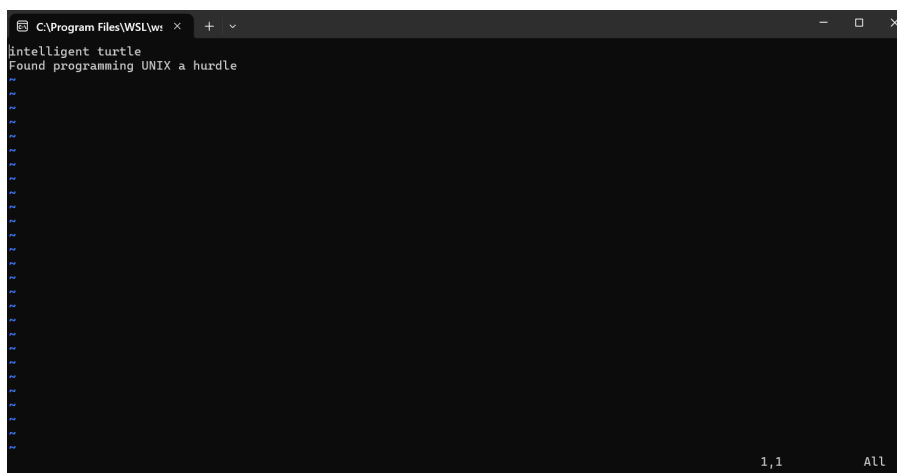
1.2.3 常见的编辑命令

1.2.3.1 移动光标 (hjkl)

回到普通模式后，你可以用 `hjkl` 代表上下左右来移动光标；这些命令看起来是随便选的。无论如何，谁听说过用 `l` 表示右的？但实际上，作这些选择是有理由的：移动光标是最常用的编辑器命令。而这些键位在你的右手本位。也就是说：这种键位的设定使你可以用最快的速度执行移动操作（特别是当你用十指输入的时候）。

1.2.3.2 删除字符

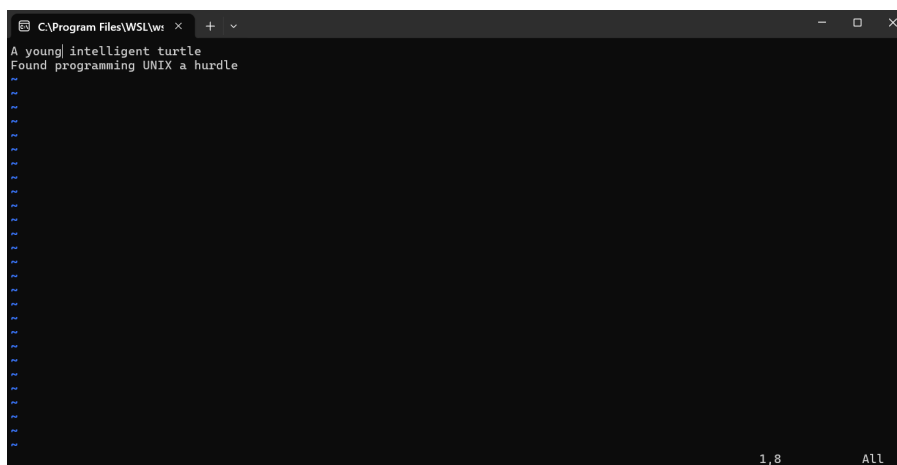
要删除一个字符，把光标移到它上面然后输入“`x`”。（这是对以前的打字机的一种回归，那时你通过在字符上输入 `xxxx` 删除它）例如，把光标移到行首，然后输入 `xxxxxxx`（七个 `x`）可以删除“A very”。结果看起来这样：



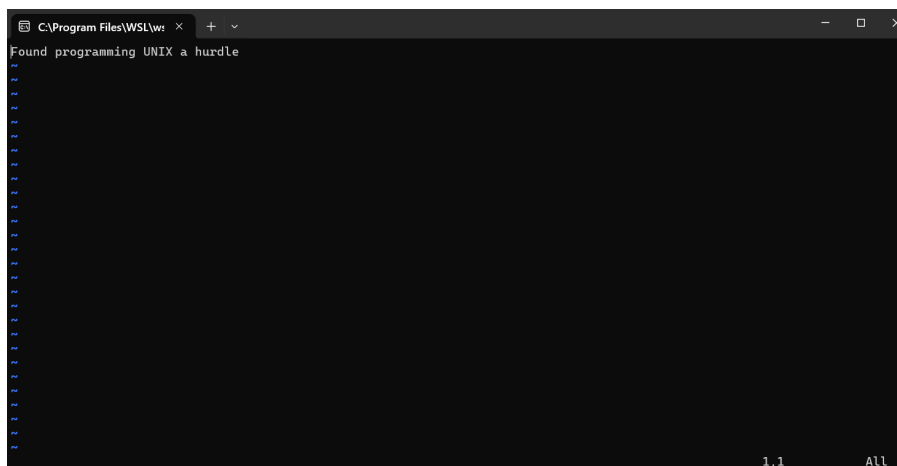
现在你可以输入新的字符了，例如，通过输入：

`iA young <Esc>`

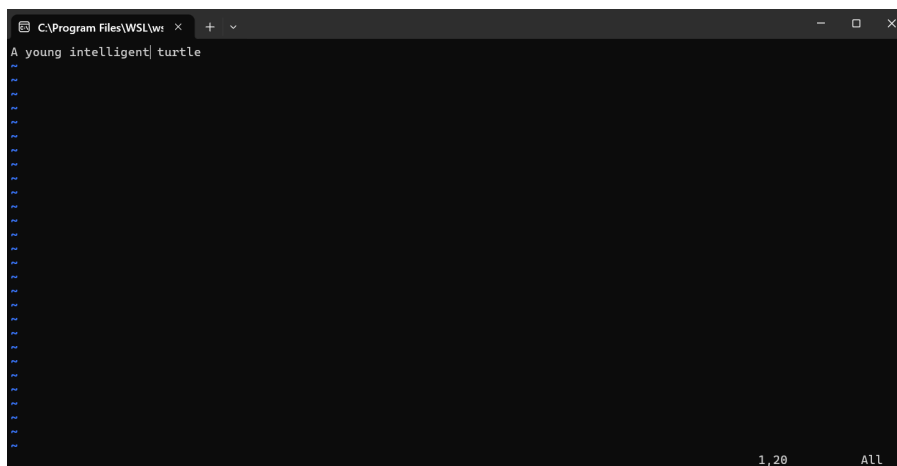
这个命令启动一次插入操作（那个“`i`”），并插入“A young”，然后退出插入模式（最后一个 `<Esc>`）。结果是：



要删除一整行，使用”dd”命令，后一行会移上来填掉留下的空行：



要删除一个换行符，或者称之为“合并行行为”，使用”J”命令；以下面两行为例，把光标移到第一行，然后按”J”：



1.2.3.3 撤销操作

编辑时难免会误删内容。与其重新输入，不如使用普通模式下的 u 命令，撤销最近一次编辑操作。例如，用 dd 删除一行后，再按 u，被删除的行就会恢复。

下面通过逐个删除字符的例子，观察撤销操作的过程。假设当前文本为：

```
A young intelligent turtle
```

现在输入 xxxxxxxx 删除“A young”。结果如下：

```
intelligent turtle
```

输入 u 撤销最后一个删除操作。该删除操作删除了字符 g，所以撤销命令恢复这个字符：

```
g intelligent turtle
```

继续按 u，文本依次恢复为：

```
ng intelligent turtle
ung intelligent turtle
oung intelligent turtle
young intelligent turtle
A young intelligent turtle
```

需要注意的是，连续按两次 `u` 并不会导致文本恢复原样，关于 `u` 的更多边界条件将在后文进一步介绍。

1.2.3.4 重做操作

如果你撤销得太多，可以输入 `CTRL-R (redo)` 回退前一个撤销命令。换句话说，它撤销一个撤销。

接着上面的例子，当文本已经完全恢复后，连续按两次 `CTRL-R`，就会重新执行对字符 `A` 及其后空格的删除操作，得到：

```
young intelligent turtle
```

除了逐次撤销，Vim 还提供了行撤销命令 `U`，用于撤销最近对当前行所做的所有改动。注意，大写的 `U` 与小写的 `u` 作用范围不同。

假设当前行为：

```
A very intelligent turtle
```

删除 `very`：

```
A intelligent turtle
```

再删除 `turtle`：

```
A intelligent
```

此时按 `U`，整行恢复到修改前的状态：

```
A very intelligent turtle
```

需要注意的是，`U` 本身也被视为一次修改。因此，再按 `u` 可以撤销这次行恢复操作，回到：

```
A intelligent
```

随后按 `CTRL-R`，又可以重做这次行恢复操作。连续使用两次 `U`，也可以取消前一次 `U` 的效果。

初学时不必急于记住所有细节。先掌握最常用的一组操作即可：误操作时用 `u` 撤销，撤销过多时用 `CTRL-R` 重做。

1.2.4 寻求帮助

学习 Vim 不需要一次记住所有命令。遇到不熟悉的操作时，可以随时查阅内置帮助。

与网络搜索相比，内置帮助有两个直接的优势：一是主题命名遵循统一规则，便于精确查找；二是帮助文档通常与所安装的 Vim 版本配套，可以减少因版本差异带来的困惑。

输入以下命令，可以打开帮助首页：

```
:help
```

如果已经知道要查找的主题，可以在命令后指定主题名称：

```
:help {主题}
```

例如，以下命令分别用于查看 `x` 命令、删除操作和命令索引的帮助：

```
:help x  
:help deleting  
:help index
```

在上文的案例中，我们已经见到了一些关于 Vim 使用的具体例子。关于命令的具体细节和边界条件，我们将在后文中详细介绍。笔者只是想借此一隅，来介绍 MiniVim 和真实 Vim 的关系，由此展现项目动机。

第 2 章 MiniVim

2.1 项目初衷

一个面向大一学生的 C++17 课程项目：从零实现一个可以在终端中运行的，具有基本 Vim(version 9.1) 操作方式的文本编辑器。

完成项目后，你应该能够：

1. 熟悉相对现代 C++ 的基本语法与 STL，如类，std::filesystem, std::optional, std::vector;(Basic)
2. 掌握基本的 **模块化设计**思想，将一个复杂问题拆分成若干职责明确，相互协作的模块；
3. 理解 **封装**的意义，通过类的公开接口隐藏内部状态和实现细节;(Advanced, 你极有可能在重构代码的时候真正体会到以上两点)
4. 对 Vim 之类的文本编辑器的底层设计实现有新的了解。(Just For Fun)

2.2 项目架构

2.2.1 框架与目录结构

MiniVim 使用 C++17，希望实现一个行为基于 Vim9.1 的终端文本编辑器。项目为 Basic 部分提供了教学框架，帮助大家把文本存储、光标移动、命令解析和终端显示拆分为职责明确的模块。当然你也可以从零设计实现，只要满足项目规定的外部行为与评测要求；对于第一次写项目的同学，请优先沿用框架。

项目中的主要文件如下。

```
MiniVim/  
|-- Makefile  
|-- *this file  
|-- src/  
|   |-- Types.hpp, Key.hpp, TextLayout.hpp  
|   |-- Buffer.hpp, Buffer.cpp  
|   |-- Window.hpp, Window.cpp  
|   |-- Command.hpp, Command.cpp  
|   |-- Renderer.hpp, Renderer.cpp
```

```
| |-- Editor.hpp, Editor.cpp
| |-- Terminal.hpp, Terminal.cpp
| |-- main.cpp
|-- test/basic/
|     |-- Note.md
|     |-- 1.in ... 28.in
|     |-- 1.ans ... 28.ans
-- docs/images/key-processing-flow.png
```

框架中直接抛出”Not implemented” 和有注释要求填写的地方都是你需要补全的函数

2.2.2 src 内各文件简要介绍

文件	模块职责
Types.hpp、Key.hpp	定义了一些基本类型。
TextLayout.hpp	辅助函数头文件, 帮助处理文本显示。
Buffer.hpp/.cpp	按行保存文件内容, 处理底层的文本修改。
Window.hpp/.cpp	作为视口, 限制合法位置并确保光标可见。
Command.hpp/.cpp	把 NormalMode 的按键解析为动作。
Renderer.hpp/.cpp	根据 Buffer,Window 的状态生成一帧的完整字符串。
Editor.hpp/.cpp	协调链接各模块成完整编辑器。
Terminal.hpp/.cpp	已提供的终端基础逻辑。
main.cpp	已提供的程序入口。

对于这些文件更加具体的描述我们作为注释写在了这些文件里, 请不要忽视这些注释

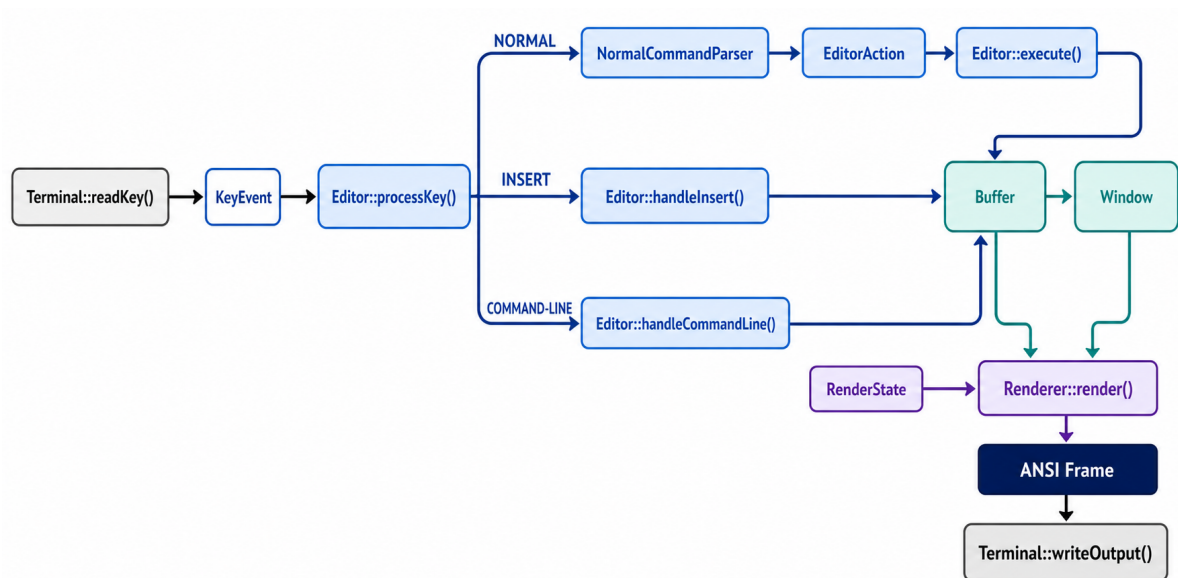
2.2.3 建议的阅读与实现顺序

阅读代码时, 建议先分文件略读, 了解各文件内接口; 再从 main 函数中的 run(), 自顶向下过一遍一次按键流入的逻辑。

实现时建议分阶段推进:

- 1. **基础数据**从简单的部分开始, 完成按键辅助函数, 实现 Buffer 来储存实际文件内容。
- 2. **光标与解析**实现 Window 的位置限制和移动, 以及 NormalModeParser 的 Basic 单键解析。
- 3. **画面生成**实现 Renderer 的帧生成逻辑。
- 4. **完整 Basic**实现 Editor, 此时可以在终端里尝试使用你的 MiniVim, 使用的过程本身就是一种 debug。
- 5. **Advanced** 此时你已经完全理解了框架, 可以自己在框架之上修改来实现 Advanced 部分的功能了

2.2.4 项目的运作流程



`Editor::ProcessKey()` 是按键分发唯一的入口。Basic 中只有 Normal 模式先经过 `NormalModeParser::Feed()` 生成 `EditorAction`, 再由 `Editor::Execute()` 执行动作。Insert 与 Command-line 分别交给 `HandleInsert()` 和 `HandleCommandLine()`。

2.3 项目开发常识

2.3.1 编译与运行

下面以 Ubuntu 24.04 环境为例：

```
sudo apt install build-essential
g++ --version
make --version
```

所有未另行说明的项目命令均在 MiniVim 根目录执行。

当前项目的 Makefile 默认设置是 `TARGET := MiniVim`, 但是提交的时候 ACMOJ 只能识别叫做“code”的二进制, 所以如果提交, 你需要改成 `TARGET := code` 接下来的讲解我们默认名字为 MiniVim, 你可以使用以下命令编译并运行：

```
make      //编译
./MiniVim //在当前根目录内运行MiniVim
./MiniVim example.txt
./MiniVim "notes with spaces.txt"
```

程序只接收零个或一个文件路径。没有路径时创建无名缓冲区；有路径时由 Buffer 按规则加载或初始化对应内容。

当你修改了 src 下文件的任何代码, 你都需要重新编译生成新的二进制
请执行：

```
make clean //清除当前的编译产物
make       //重新编译
```

2.3.2 Basic 部分的 Debug

完成 Basic 后，可按以下顺序进行一次人工检查：

1. 启动程序，就像使用一个真正的文本编辑器一样自由使用你已经实现过的 Basic 功能。

如果你对于你的 MiniVim 在这些操作都很满意，那你就应该有自信认为你的程序正确在你有自信的情况下，可以使用我们下发的 basic 部分测试点进行最终确认，我们保证这些测试点和 ACMOJ 上的 Basic 测试点要求是完全相同的

2.4 项目测评

下发测试 Basic 部分的全部测试点，具体使用说明在 test/basic/Note.md 里

在线评分 Basic 最终得分以 ACMOJ 评测为准。Advanced 部分测试点我们不会下发，但我们保证测评内容不会超出 spec 的要求。

2.4.1 项目测评工具 vtemu 的使用与介绍

2.4.1.1 工具作用与准备

vtemu 是终端交互程序的脚本驱动器和屏幕采样器。它通过伪终端 (PTY) 运行 MiniVim，将.in 文件的输入序列转换为按键传给 MiniVim，再不断将虚拟终端的截图转化成文本序列，输出到.ans 文件中。

这样，我们就可以利用重定向输入输出流来自动化评测你的 MiniVim 了！

具体使用的方式请见 basic/test/Note.md

但在使用之前，我们需要先编译它，先切换到 vtemu 的根目录

```
make all// 编译vtemu
```

你应该在/vtemu/bin 下发现一个叫做 vtemu 的二进制文件

2.4.1.2 脚本记号与采样

记号	含义
<SP>、<TAB>	空格、Tab
<CR>、<BS>	Enter、Backspace。
<ESC>	Esc
<X>、<X300>	等待默认或指定时间
<EX>	Esc 再等待默认时间。
<P-0;0;1;80>	输出屏幕矩形 [top, left, bottom, right] 区域的文本快照
<CURP>	输出光标坐标
<E>	结束驱动器并清理子进程

希望这些内容可以帮助你理解我们的下发测试点 (你会发现我们的测试点都是很仁慈的!)

2.5 项目实现要求

从此开始就进入了我们对于项目实现的指导，本项目 Basic 部分上限 60 pts, Advanced 上限 40pts, Extra 上限 10pts, 总分上限为 100pts, 不溢出; Code Review 期间如果发现你对自己的代码了解不足，甚至不清楚自己在写什么，会对应部分功能代码的分数采取清零措施。

2.5.1 Basic 部分要求

你必须实现 Basic 部分才能获得基础分数并继续完成 Advanced 部分与 Extra 部分. Basic 部分的最终得分即为 ACOJ 上评测得到的分数.

2.5.1.1 Basic 概述

MiniVim 的哲学是模态编辑, 高效的编辑操作依托各模式间的切换完成. 简而言之, 在 Basic 部分中, 你需要为 MiniVim 实现最基础的 3 种模式:

- **Normal:** 移动光标, 执行编辑动作或进入其他模式;
- **Insert:** 插入和删除文本;
- **Command-line:** 输入保存, 退出等以 : 开始的命令;

并为每个模式实现一些最基础的功能.

- 在 Basic 部分, 你可以简单地认为屏幕由两部分构成: 屏幕最下面一行的保留部分 (用于输入命令, 显示信息等), 以及上面其余的编辑部分, 即缓冲区.
- Normal 模式与 Insert 模式主要操作缓冲区, 而 Command-line 模式则会将保留部分当成一块用于输入命令的命令缓冲区来操作.
- 在 Basic 部分中, 所有要求的功能在终端上打印的字符都以默认颜色与属性呈现 (具体来说, 都使用默认前景色与背景色; 没有额外终端字符属性, 如粗体). 也就是说, 你不需要操作任何颜色与属性即可通过 Basic 评测.
- <C-q> 在该项目中是未被绑定的保留键位. 你可以将其实现为紧急退出键, 按下后在任意模式下结束程序, 以便方便调试 Basic 及之后的部分. 我们不会对该键行为进行评测.

2.5.1.2 Normal 模式 (Basic)

在 Normal 模式中, 光标总会停留在某一个字符上 (对于空行, 则只能停留在不存在字符的位置).

在 Basic 部分, 你需要为 Normal 模式实现以下的功能键.

1. 光标移动: h, j, k, l, 且不要求实现其他额外功能, 详见 4.1
 2. 文本的修改: 实现 i, a, 详见 7.1
 3. Command Mode: 实现 :, 要求能够进入 Command Mode, 详见 6
- 请注意对于空行, 该行的行末位置不存在实际字符; 而对于非空行, 该行的行末位置是最后一个字符的位置.
 - 对于以上的移动的 curswant 功能, 不要求在 Basic 部分中准确实现, 但是在 Advanced 部分中会进行评测. 具体关于该功能的说明详见 curswant.

2.5.1.3 Insert 模式 (Basic)

在 Insert 模式中, 光标表示字符之间的插入位置: 在某字符之前, 或在最后一个字符之后. 进入 i 使用当前列; 进入 a 时非空行使用 column + 1, 空行仍为 0. 不在 Buffer 行内额外存储占位换行符.

在 Basic 部分, 你需要为 Insert 模式实现以下的功能键:

文本的修改: 实现 ascii 码, Backspace, Enter, Esc 键位的正常键入: {printable ascii}, <BS>, <CR>, <Esc>, 详见 7.1

2.5.1.4 Command-line 模式 (Basic)

在 Basic 部分, 你需要为 Command-line 模式实现以下的功能键:

命令的键入: 实现 ascii 码, Backspace, :wq, :q! : {printable ascii}, <BS>, <CR>, <Esc>, 详见 6

- 若当前输入的指令为空字符串, 则不会有任何行为.

- 若当前输入的指令非法, 在 Basic 部分中你可以直接忽略执行它. 在 Basic 部分中, 我们不会对未提及的命令评测.

2.5.2 Advanced 部分要求

Advanced 部分包含若干可选功能。它们之间大部分相互独立，但有些功能存在依赖关系，这意味着部分功能的实现需要其他功能来支持；建议按照本 README 的行文顺序来实现或许会更为方便。

对于 Advanced 部分，你可以选择任意一部分功能来实现。

- 每个功能在 ACMOJ 中都有独立测试点计分，ACMOJ 给出的总得分在折算之后作为 Advanced 部分的实际得分。折算公式为 $y = f(x)$ ，其中 x 为你的 ACMOJ 计分， y 为你的实际得分。
- 如果你使用我们提供的框架，从本部分开始，除 Basic 中已经给出的公共接口与 Terminal 外，不再限制 src/ 中其他模块的接口设计。
- 你可以添加新的类、数据结构、public/private 接口或源文件，但已经完成的 Basic 行为不得被破坏。
- 除非特别说明，本节中的命令语义按照 Vim 的行为定义。ACMOJ 的评测不会测试未提及的 Vim 行为与特性。若下文明确给出与 Vim 默认行为不同的要求，则以该要求为准。
- 有些功能的实现与大量操作相关联（如 [count]{operation}，许多操作都支持加上计数），则该功能的评分会在所有所关联的操作下独立评分，详见给分细则。

在这一部分你需要实现：

2.5.2.1 光标移动

1. h, j, k, l, 详见 4.1
2. O, \$, ^, 详见 ??
3. gg, G:, 详见 ??
4. w, e, b, ge, W, E, B, GE, 详见 ??
5. %, 详见 ??
6. m, :, 详见 ??
7. \, ?, n, N, 详见 ??

2.5.2.2 文本编辑

1. i, I, a, A, o, O, Del, 详见 7.1
2. x, X, 详见 ??
3. d, dd, D, x, X, 详见 ??
4. c, C, 详见 ??
5. r, 详见 ??
6. <, >, <<, >>, 详见 ??
7. y, p, P, 详见 ??
8. ", q, @, 详见 ??
9. ~, 详见 ??
10. J, 详见 ??
11. ., 详见 ??
12. u, C-r, 详见 ??

2.5.2.3 模式切换

1. :, 详见 6

2. v, 详见??
3. ESC, 详见??与??

2.5.2.4 文件操作

1. :w, 详见6.3
2. :q, 详见6.3
3. :wq, 详见6.3
4. :edit, 详见6.3

2.5.2.5 窗口操作

zt,zz,zb,<C-e>,<C-y>,<C-d>,<C-u>, 详见??

2.5.2.6 评分标准

本部分总分共 300 pts, 按照公式换算为至多 40pts. 公式为:

$$f(x) = \begin{cases} \frac{1}{3}x, & 0 \leq x \leq 30, \\ \frac{1}{6}x + 5, & 30 < x \leq 90, \\ \frac{1}{9}x + 10, & 90 < x \leq 180, \\ \frac{1}{12}x + 15, & 180 < x \leq 300. \end{cases}$$

表 2.1: Vim 按键功能对照及评分表

实现功能	分数及评分细则
h, j, k, l	6 pts curswant: 2 pts; [count] h,l: 2 pts; [count] j,k: 2 pts
0, \$, ^	10 pts 0: 2 pts; \$: 3 pts; [count] \$: 2 pts; ^: 3 pts
gg, G	6 pts gg: 2 pts; G: 2 pts; [count] gg / G: 1+1 pts
w, e, b, ge	12 pts w,e,b,ge: 8 pts; [count] w,e,b,ge: 4 pts;
W, E, B, gE	8 pts W,E,B,gE with count: 8 pts;
%	5 pts %: 3 pts; [count] %: 2 pts
m, `	7 pts m, `: 3 pts; m with edits: 4 pts
\, ?, n, N	10 pts \, ?: 3 + 3 pts; n, N: 1 + 1 pt; [count] n/ N: 1 + 1pt

续下页

表 2.1: Vim 按键功能对照及评分表

实现功能	分数及评分细则
i, I, a, A, o, O, Del	20 pts i, a with curswant: 2+2 pts; [count] i, a, I, A, o, O: 2+2+2+2+2+2 pts; Del: 4 pts
x, X	4 pts x: 2 pts; X: 2 pts
d, dd, D	20 pts d charwise: 6 pts; d linewise: 4 pts; dd, D: 2+2 pts; d, dd, D with count: 2+2+2 pts
c, cc, C	20 pts c charwise: 6 pts; c linewise: 4 pts; cc, C: 2+2 pts; c, cc, C with count: 2+2+2 pts
r	3 pts
<, >, <<, >>	10 pts <, >: 4 pts; <<, >>: 2 pts; with count: 2+2pts
y, p, P	34 pts y charwise: 6 pts; y linewise: 4 pts; yy, Y: 2+2 pts; y, yy, Y with count: 2+2+2 pts p, P charwise: 4 + 4 pts; p, P linewise: 3+3 pts
" , q, @	16 pts ": 4 pts; q: 6 pts; Q, @: 6 pts
~	2 pts
J	4 pts
.	7 pts
u, Ctrl-r	16 pts u: 4 pts; Ctrl-r: 8 pts; U: 4 pts
Visual Mode	26 pts basic motions: 10 pts; operators d, c, y with count: (2+2+2)*2 = 12 pts <, >: 2 pts; ~: 2 pts;
textobjects	30 pts
:w, :q, :wq, :edit	8 pts :q, :q!: 2 pts;
:w, :w!: 2 pts; :wq, :wq!: 2 pts; :e, :e!: 2 pts	
scroll	6 pts

续下页

表 2.1: Vim 按键功能对照及评分表

实现功能	分数及评分细则
zt, zz, zb	6 pts 2+2+2 pts
<C-e>, <C-y>, <C-d>, <C-u>	4 pts 1+1+1+1 pts

2.5.3 Extra 部分要求

土鳖康铁牛.....

第 3 章 项目前置知识

3.1 按键记号约定

文档中常使用 <> 记法标注按键。有时代表实际输入键 (为清晰起见)，多数场景下代表按本义输入序列, 用 <key> 标记输入的特殊键。对应下表的按键名称。

表 3.1: 按键名称与含义对照表

按键名称	含义
<BS>	退格键
<Tab>	制表符
<CR>	回车键
<Esc>	转义键
<Space>	空格键
	删除
<S-...>	组合 Shift 键, shift+...
<C-...>	组合 Control 键, ctrl+...
<C-G>	组合 Control 键, ctrl+G

第 4 章 光标的移动 (一)

4.1 上下左右移动:hjkl

1. h: 光标向左移动一个字符; 如果光标在行首则不变.
2. l: 光标向右移动一个字符; 如果光标在行末则不变.
3. j: 光标向下移动一条文本行; 如果光标在最后一行则不变. 目标行过短时停在其最后字符, 空行停在第 0 列.
4. k: 光标向上移动一条文本行; 如果光标在第一行则不变. 列的锚制规则与 j 相同.

第 5 章 光标的移动 (二)

第 6 章 Command-line 模式与文件操作

6.1 Command-line 模式

在 Command-line 模式中,

1. 光标总会停留在窗口底部的命令输入部分, 总体操作逻辑类似于 Insert 模式中的一行, 并且此时行首必定以: 开头.
2. 相较于 Normal 模式的状态, 除了屏幕最下面一行, 其余部分显示内容应保持不变.
3. 进入到 Command-line 模式时, 初始输入的指令, 即命令缓冲区部分为空.
4. 在 Basic 部分中, 你只需要考虑光标处在行末位置的情形. 也就是说, 任何停留在 Command-line 模式的操作最终都会将光标移动到行末位置.

特别地,

- 若当前输入的指令为空字符串, 则不会有任何行为.
- 若当前输入的指令非法, 在 Basic 部分中你可以直接忽略执行它. 在 Basic 部分中, 我们不会对未提及的命令评测.

6.2 基本按键

6.2.1 printable ascii(Basic)

- 在光标所选字符之前 (在 Basic 部分中, 即行末) 插入输入的字符, 并将光标自然移动到下一个位置上 (在 Basic 部分中, 即行末).
- 这部分功能键限定为所有 ASCII 码在 0x20 0x7e 之间的字符.

6.2.2 <CR>(Basic)

- 执行当前命令缓冲区中的指令, 待执行完毕后进入 Normal 模式.

6.2.3 <Esc>(Basic)

- 进入 Normal 模式, 并将光标移动到进入 Command-line 模式前原来的位置. 舍弃在 Command-line 模式中的所有输入.

6.3 文件操作

6.3.1 :wq [file] (Basic)

没有任何编辑操作的情况下（不包含移动操作）文件里没有多余的换行

1. 将缓冲区内容写入到该路径所在文件.
2. 若文件不存在, 则创建新文件并写入.
3. 若文件已存在, 则覆盖原有内容.
4. 在 Basic 部分中不需要考虑文件权限等问题.

6.3.2 :q!, :quit! (Basic)

舍弃缓冲区内容并退出 MiniVim.

6.4 Command-line Mode 显示要求

不计分, 但是建议实现

- 实现相应模式时显示 -- INSERT --、-- VISUAL --、-- REPLACE -- 等提示; 命令行输入期间只显示命令行.
- Buffer 之外的填充行以 ~ 标识; Advanced 使用亮蓝色.
- Basic 使用默认前景色、背景色和字符属性, 不要求额外颜色.

第 7 章 文本的修正与移动 (一)

7.1 插入

7.1.0.1 i

在光标所选字符之前进入 Insert 模式. 若光标处在空行, 则在该空行的唯一位置进入 Insert 模式.

7.1.0.2 a

在光标所选字符之后进入 Insert 模式. 若光标处在空行, 则在该空行的唯一位置进入 Insert 模式.

7.1.0.3 {printable ascii}

- 在光标所选字符之前插入输入的字符, 并将光标自然移动到下一个位置上.
- 这部分功能键限定为所有 ASCII 码在 0x20~0x7e 之间的字符.

7.1.0.4 <Tab>

输入单个 0x09, 不展开成空格; 其他未列出的特殊键忽略. 编辑后立即用新行内容重算显示列.

7.1.0.5 <BS>

- 删除光标所选字符之前的字符, 并将光标自然移动到前一个位置上.
- 若光标处在行首 (或空行), 则拼接该行与上一行, 并将光标自然移动到上一行原行末位置上.
- 若光标处在文件首, 则无效果.

7.1.0.6 <CR>

- 在光标所选字符之前位置切分该行, 换行至下一行, 并将光标自然移动到下一行的行首.
- 行首、行尾、空行均允许拆行, 不自动缩进. 更新 Buffer 后设置 (row + 1, 0), 不复制原 curswant 到新行.

7.1.0.7 <Esc>

- 进入 Normal 模式, 并将光标自然移动到前一个位置上. 若光标处在行首则不移动光标.
- 有插入 count 时先完成回放; 然后仅当列大于 0 才左移一次, 不跨行; 最后按 Normal 范围钳制并以实际列更新 curwant. i<Esc> 可以只改变光标而没有编辑节点, 不要因此伪造 modified. 历史录制见插入重放.